Testing de la API del Proyecto MONK GRAIN con Postman

Análisis y Desarrollo de Software

Servicio Nacional de Aprendizaje (SENA)

Eliana Chacon

1 de noviembre de 2026

1. Introducción

Esta evidencia prueba con Postman la API del proyecto formativo documentada en AA5-EV03: 107 endpoints en 12 grupos.

Probar los 107 uno por uno daría una colección enorme y poco útil. La colección recorre en cambio **tres caminos completos** —el del comprador, el del vendedor y el de la autorización— encadenando los datos: el identificador de la variante sale del catálogo, el del renglón sale del carrito, la fecha de vencimiento se calcula. No hay nada escrito a mano.

28 peticiones, 48 comprobaciones. Y la parte más útil de la evidencia no es que pasen: es que en el primer intento **fallaron cuatro**, y cada fallo enseñó algo distinto. Dos eran errores míos y dos eran defectos reales del proyecto. La sección 7 los cuenta.

2. Objetivo

Probar con Postman las API construidas en AA5-EV03: instalar la herramienta, construir la colección, documentar con pantallazos, grabar el video del testing y entregar el archivo de endpoints.

3. Preparación

```
# 1. levantar el proyecto completo
docker compose up -d

# 2. comprobar que responde
curl -s http://localhost:8000/health
{"status":"ok"}

# 3. importar en Postman
#    entregas/GA7/AA5_EV04_Postman_Proyecto/postman/
#        MONK_GRAIN_API_Proyecto.postman_collection.json
```

Captura 1 — El proyecto en marcha

Terminal con `docker compose ps` mostrando los seis servicios (`backend`, `web`, `postgres`, `redis`, `minio`, `mailpit`) en estado running

Captura 2 — La colección importada

Postman con «MONK GRAIN — API del proyecto (AA5-EV03)» y sus cinco carpetas

3.1 Las cuentas de prueba

Variable	Valor	Rol
`correoComprador`	`trilladora.popayan@ejemplo.co`	`consumer`
`correoVendedor`	`finca.elmirador@ejemplo.co`	`producer`
`claveDemo`	`MonkGrain2026Demo`	—

La primera de las dos **no existía** hasta que esta colección la echó de menos. Está contado en §7.2.

4. La colección

Carpeta	Peticiones	Qué recorre
1. Salud y autenticación	5	Que el servicio esté vivo y las dos cuentas entren
2. Catálogo	4	Lista, filtro por factor, detalle con variantes
3. Carrito	8	La regla del mínimo de kilos, por los dos caminos
4. Oferta y demanda	5	El comprador publica, el vendedor lo ve
5. Autorización	**6**	**Lo que la API NO debe dejar hacer**

Todo lo que hace falta entre peticiones viaja en variables que las propias pruebas escriben:

```
1.2  →  tokenComprador          1.3  →  tokenVendedor
2.1  →  slugProducto            2.3  →  idVariante, minimoKg
3.3  →  idItemCarrito           4.1  →  idSolicitud
```

Ninguna petición lleva un identificador escrito a mano. Si los llevara, la colección dejaría de funcionar la primera vez que alguien recargara los datos de demostración.

5. Lo que comprueba cada carpeta

5.1 Salud y autenticación

`/health` es el único endpoint fuera de `/api/v1`, y una prueba lo fija a propósito: el orquestador que comprueba si el servicio está vivo no debe tener que saber en qué versión va la API, porque entonces migrar a `/v2` tumbaría la comprobación de salud antes que ninguna otra cosa.

El ingreso devuelve **dos** tokens, y la prueba exige los dos:

```
pm.test('Entrega los dos tokens', function () {
    const c = pm.response.json();
    // Dos y no uno: el de acceso dura 30 minutos y viaja en cada
    // petición; el de renovación dura mucho más y solo viaja al renovar,
    // así que se expone muchas menos veces.
    pm.expect(c.access_token).to.be.a('string').and.not.empty;
    pm.expect(c.refresh_token).to.be.a('string').and.not.empty;
});
```

Y `/auth/me` comprueba que vengan los **permisos** y no solo el rol: la API autoriza por permiso, hay 24 de ellos, y el cliente necesita la lista para decidir qué botones mostrar.

Captura 3 — Ingreso del comprador (1.2)

La respuesta `200` con los dos tokens, y las tres comprobaciones en verde

5.2 Catálogo

Tres comprobaciones que fijan reglas del dominio, no solo códigos de estado:

```
pm.test('Ningún lote pasa del factor pedido', function () {
    // El filtro es max_ y no min_, y eso resume el dominio: en el factor
    // de rendimiento MENOS ES MEJOR. El estándar de la Federación es 94,
    // así que pedir «92 o mejor» es pedir café por encima del estándar.
    pm.response.json().items.forEach(function (lote) {
        pm.expect(Number(lote.yield_factor)).to.be.at.most(92);
    });
});
```

```
pm.test('El precio viaja como cadena, no como número', function () {
    // Un float no representa exactamente 0,1. Sobre una carga de 125 kg,
    // un redondeo mal hecho son miles de pesos de diferencia entre lo
    // que el comprador ve y lo que el vendedor cobra.
    pm.expect(pm.response.json().items[0].base_price).to.be.a('string');
});
```

Y una que fija la envoltura de página —`items`, `total`, `page`, `page_size`, `total_pages`, `has_next`, `has_previous`— con `to.have.all.keys`, de modo que añadir o quitar un campo rompe la prueba. Es la misma forma en las once listas de la API: un cliente que la aprende una vez sabe recorrerlas todas.

Captura 4 — Filtro por factor de rendimiento (2.2)

La respuesta con los lotes de factor 92 o mejor, y la comprobación que recorre todos los resultados en verde

5.3 Carrito: la regla del mínimo, por los dos caminos

El pergamino se vende desde un mínimo —250 kg en el lote de la prueba— y no se despacha un kilo suelto. La regla vive en el servidor: si solo estuviera en el formulario, bastaría con hablarle a la API directamente para saltársela.

El orden de esta carpeta importa más que en las demás:

#	Petición	Esperado	Por qué
3.1	Vaciar el carrito	`200/204`	Punto de partida limpio
3.2	Un kilo, carrito **vacío**	`422`	Aquí la regla tiene que actuar
3.3	Añadir el mínimo	`201`, 250 kg	—
3.4	`GET /cart`	un renglón	—
3.5	Un kilo **más**	`201`, 251 kg	Suma: 251 sigue respetando el mínimo
3.6	`PATCH` a 1 kg	`422`	No basta validar al añadir
3.7	Variante inventada	`404/422`	—
3.8	Sin token	`401`	El carrito es de quien lo presenta

3.2 y 3.5 envían **exactamente el mismo cuerpo** —un kilo de la misma variante— y esperan cosas distintas, porque el estado del carrito es distinto. Esa pareja es la que hace honesta a la carpeta: la primera versión de la colección probaba el kilo suelto después de llenar el carrito y daba por defectuoso un comportamiento correcto (§7.1).

3.6 es la otra mitad de la regla, y la que más se olvida: validar solo al añadir deja abierta la vía de modificar la cantidad después.

Captura 5 — Un kilo en el carrito vacío (3.2)

La respuesta `422` con `"Este lote se vende desde 250 kg."` y `details.min_qty_kg`

Captura 6 — El mismo kilo, ya con el mínimo dentro (3.5)

La respuesta `201` con la cantidad en 251. Misma petición que la captura anterior, resultado distinto porque el estado es distinto

Captura 7 — Bajar por debajo del mínimo (3.6)

El `PATCH` a un kilo rechazado con `422`

5.4 Oferta y demanda

Un recorrido corto por el mercado en su segundo sentido: el comprador publica una solicitud de 4.000 kg con factor 92 o mejor, comprueba que aparece en la lista y en el detalle con su cobertura en cero, y **el vendedor la ve desde su propia ruta**. Al final se cierra, para no dejar solicitudes de prueba abiertas en los datos de demostración.

La fecha de vencimiento se calcula en el guion previo:

```
// El vencimiento se calcula, no se escribe: una fecha fija en el cuerpo
// haría que la colección dejara de funcionar el día que pasara.
const d = new Date();
d.setDate(d.getDate() + 30);
pm.collectionVariables.set('vencimiento', d.toISOString().slice(0, 10));
```

Y el detalle comprueba que lleguen **los kilos pedidos y los cubiertos**, no solo el porcentaje: «80 %» de una solicitud pequeña puede ser menos café que el 20 % de una grande.

Captura 8 — Solicitud publicada (4.1)

La respuesta `201` con `status: "open"` y `covered_kg: 0`

Captura 9 — La misma solicitud vista por el vendedor (4.4)

`GET /producer/demands` con el token del vendedor, y la solicitud del comprador en la lista

5.5 Autorización: lo que la API no debe dejar hacer

La carpeta más importante, y la que no suele aparecer en una colección de ejemplo. Un endpoint que funciona para quien tiene permiso y también para quien no, funciona mal.

#	Quién	Contra qué	Esperado
5.1	Comprador	Panel del productor	`403`
5.2	Comprador	Administración	`403`
5.3	Vendedor	Administración	`403`
5.4	Comprador	Un pedido ajeno	**`404`**
5.5	Token alterado	`/auth/me`	`401`
5.6	Esquema `Basic`	`/auth/me`	`401`

5.1 a 5.3 exigen **403 y no 401**. La distinción importa: el usuario sí está identificado, lo que le falta es el permiso. Confundirlos hace que el cliente mande a iniciar sesión a alguien que ya tiene la sesión abierta.

5.4 exige lo contrario de lo que parece correcto:

```
pm.test('Responde 404', function () {
    // Deliberadamente 404 y no 403. Un 403 confirmaría que ese pedido
    // existe, y eso ya es información: permitiría recorrer
    // identificadores y contar cuántos pedidos tiene la plataforma. El
    // 404 no distingue «no existe» de «no es tuyo», que es exactamente
    // la ambigüedad que conviene.
    pm.response.to.have.status(404);
});
```

5.5 no usa un token inventado sino el token bueno **con un carácter cambiado**, justo en la firma. Es un caso distinto: comprueba que el servidor verifica la firma y no solo la forma.

Captura 10 — El comprador contra el panel del productor (5.1)

La respuesta `403` con `error.code`

Captura 11 — Un pedido ajeno (5.4)

La respuesta `404`, no `403`

6. Ejecución completa

```
$ newman run MONK_GRAIN_API_Proyecto.postman_collection.json

┌─────────────────────────┬────────────────────┬────────────────────┐
│                         │           executed │             failed │
├─────────────────────────┼────────────────────┼────────────────────┤
│              iterations │                  1 │                  0 │
│                requests │                 28 │                  0 │
│            test-scripts │                 28 │                  0 │
│      prerequest-scripts │                  2 │                  0 │
│              assertions │                 48 │                  0 │
├─────────────────────────┴────────────────────┴────────────────────┤
│ total run duration: 1446ms                                        │
│ total data received: 18.95kB (approx)                             │
│ average response time: 29ms [min: 6ms, max: 113ms, s.d.: 24ms]    │
└───────────────────────────────────────────────────────────────────┘
```

Captura 12 — El Collection Runner terminado

El resumen con las 48 comprobaciones en verde y ningún fallo

Si se ejecuta tres o cuatro veces seguidas, 1.2 y 1.3 responden 429. No hay nada que arreglar: el ingreso admite 10 intentos por minuto y por cuenta, y la colección está agotando ese tope. Basta esperar un minuto. La colección lo dice con palabras en lugar de dejar un «expected 200 but got 429» sin explicación:

```
if (pm.response.code === 429) {
    pm.test('⚠ Tope de frecuencia alcanzado: espera un minuto y vuelve a ejecu-
tar', ...);
}
```

7. Los cuatro fallos del primer intento

Esta es la sección que justifica la evidencia. En la primera ejecución, 4 de las 42 comprobaciones fallaron. Investigarlas una por una dio dos errores propios y dos defectos reales.

7.1 Mi prueba estaba mal: el kilo suelto

La prueba decía «se rechaza un kilo suelto» y recibió un `201`. Parecía un agujero en la validación. No lo era: la prueba corría después de añadir 250 kg, y sumar un kilo a un renglón que ya tiene el mínimo lo deja en 251, que sigue respetando la regla. Comprobado a mano:

```
añadir 250  →  cantidad en el carrito: 250
añadir   1  →  cantidad en el carrito: 251      ← suma, no reemplaza
PATCH a  1  →  422  "Este lote se vende desde 250 kg."
```

La API era correcta y la prueba estaba mal diseñada. El arreglo fue reordenar la carpeta: el kilo suelto se prueba sobre el carrito **vacío**, y el incremento y el `PATCH` por debajo del mínimo se prueban aparte. La carpeta quedó mejor que antes: tres comprobaciones donde había una.

7.2 Defecto real: no había ninguna cuenta de comprador

La prueba «el comprador entra al panel del productor → 403» recibió un `200`. Parecía un agujero de autorización, que sería grave. Mirando la cuenta:

```
$ curl .../auth/me -H "Authorization: Bearer $TOKEN"
roles:    [{"code": "producer", "name": "Productor"}]
permisos: ["inventory.manage", "order.manage_sales", "product.create", ...]
```

La cuenta que yo había tomado por comprador — `tostaduria.monje@ejemplo.co` — era de **productor**. El 200 era correcto. Y al revisar los datos de demostración apareció el defecto de fondo:

```
        email                   |  roles
--------------------------------+----------
 admin.verificacion@...         | admin
 coop.narino@ejemplo.co         | producer
 finca.elmirador@ejemplo.co     | producer
 finca.laesperanza@ejemplo.co   | producer
 hacienda.altavista@ejemplo.co  | producer
 tostaduria.monje@ejemplo.co    | producer
```

Las cinco cuentas de la semilla eran de productor, incluida la tostaduría, que en este mercado compra pergamino y no lo vende. Para un mercado de dos lados, los datos de demostración no permitían demostrar la mitad del producto: quien clonara el repositorio no podía entrar como comprador a recorrer catálogo, carrito y pedido.

Se corrigió añadiendo la cuenta que faltaba, con su propia función en `demo_data.py` —un comprador no tiene perfil de productor ni finca— y validando el código DANE del municipio contra el catálogo:

```
$ docker compose exec backend python -m app.db.demo_data

Datos de demostración cargados:
  productores : 5
  compradores : 1 nuevos
  ...
Acceso de prueba:
  vendedor   : finca.elmirador@ejemplo.co
  comprador  : trilladora.popayan@ejemplo.co
  contraseña : MonkGrain2026Demo

$ curl .../producer/products -H "Authorization: Bearer $TOKEN_COMPRADOR"
403
```

7.3 Defecto real: la documentación mentía sobre los errores

La prueba esperaba un campo `detail`, como documentaba AA5-EV03, y recibió:

```
{"error": {"code": "not_found", "message": "No encontramos este café."}}
```

La API no usa el formato de serie de FastAPI: un manejador propio lo envuelve con un código legible. El documento de AA5-EV03 afirmaba `{"detail": "..."}` porque es lo que se supone sin mirar, y nadie lo había contrastado. Se corrigieron las dos cosas: el apartado 4.5 de aquel informe y la tabla de convenciones de `ENDPOINTS.md`.

Es el hallazgo más instructivo de los cuatro. Una afirmación **plausible y falsa** sobrevive a cualquier revisión de lectura: nadie se detiene a verificar lo que ya parece correcto. Solo la rompe una prueba que vaya a preguntarle al servicio.

7.4 Mi prueba estaba mal: el nombre de un campo

«El carrito no está vacío» fallaba porque buscaba los renglones en `items` o `sellers`, y la respuesta real los trae en `groups[].items[]`. Había adivinado el nombre en vez de mirarlo. El arreglo, además de corregirlo, convirtió el error en una comprobación nueva: que el carrito **venga agrupado por vendedor**, que es lo que hace posible que un pedido con lotes de tres fincas genere tres sub-órdenes.

7.5 Resumen

Fallo	De quién	Qué dejó
El kilo suelto	De la prueba	La carpeta del carrito, con 3 comprobaciones donde había 1
El comprador que era productor	**Del proyecto**	La cuenta de comprador que faltaba en la semilla
El formato de error	**Del proyecto**	El apartado 4.5 de AA5-EV03, corregido
El nombre del campo	De la prueba	Una comprobación nueva: el carrito agrupa por vendedor

8. Guion del video

Entre ocho y diez minutos. El proyecto debe estar en marcha antes de grabar: `docker compose up -d`.

Bloque	Minutos	Qué mostrar y qué decir
1. Presentación	0:00 – 0:45	Nombre, programa, y qué se va a probar: la API del proyecto MONK GRAIN, 107 endpoints. Decir que la colección no los prueba uno por uno sino que recorre tres caminos completos.
2. El proyecto y la colección	0:45 – 1:45	`docker compose ps` con los seis servicios. Abrir `http://localhost:8000/docs` y mostrar los 12 grupos. Luego Postman con la colección y las variables, señalando que ningún identificador está escrito a mano.
3. Salud y autenticación	1:45 – 3:00	Ejecutar 1.1 y señalar que `/health` no lleva versión, y por qué. Luego 1.2 y 1.3: los dos tokens, y por qué dos. Y 1.4: que `/auth/me` trae los permisos, no solo el rol.
4. Catálogo	3:00 – 4:00	Ejecutar 2.1 y mostrar la envoltura de página. Luego 2.2 y **detenerse ahí**: el filtro es `max_yield_factor`, porque en el factor de rendimiento menos es mejor. Es el dato que resume el proyecto.
5. El carrito y la regla del mínimo	4:00 – 5:45	Ejecutar 3.2 con el carrito vacío: `422`, se vende desde 250 kg. Luego 3.3 y 3.5, y señalar que 3.5 envía el mismo cuerpo que 3.2 y se acepta, porque 251 sigue respetando el mínimo. Cerrar con 3.6: bajar a un kilo se rechaza.
6. Oferta y demanda	5:45 – 7:00	Ejecutar 4.1, la solicitud de 4.000 kg. Luego 4.4 con el token del vendedor: la misma solicitud, vista desde el otro lado. Decir aquí para qué sirve el estado «parcialmente cubierta».
7. Autorización	7:00 – 8:30	Ejecutar 5.1 y 5.2: `403`, no `401`, y explicar la diferencia. Luego 5.4 y detenerse: `404` y no `403` a propósito, porque un 403 confirmaría que ese pedido existe.
8. La colección completa	8:30 – 9:15	Collection Runner de principio a fin: 48 comprobaciones en verde.
9. Cierre	9:15 – 10:00	Contar los dos defectos reales que encontró la colección (§7.2 y §7.3) y decir dónde está todo: la colección en `postman/` y el catálogo en `ENDPOINTS.md`.

> **Lo que te falta hacer a ti**
>
> **Las doce capturas** señaladas a lo largo del documento, cada una con lo que debe verse descrito al lado.
>
> **El video de 8 a 10 minutos**, con el guion de la sección 8.
>
> Para las dos cosas basta `docker compose up -d` e importar la colección. Ya está construida y verificada; las cuentas de prueba están en la sección 3.1 y la contraseña es la misma para las dos. Si al ejecutarla dos veces seguidas aparece un `429` en el ingreso, espera un minuto: es el tope de frecuencia, no un fallo.

9. Conclusión

Las 28 peticiones pasan sus 48 comprobaciones, y el recorrido cubre los grupos principales de los 107 endpoints encadenando datos reales en lugar de identificadores escritos a mano.

Pero lo que esta evidencia demuestra de verdad es otra cosa. De los cuatro fallos del primer intento, dos eran defectos del proyecto que llevaban ahí desde antes: unos datos de demostración que no permitían demostrar la mitad del producto, y un documento de diseño que afirmaba un formato de error que la API no usa. Ninguno de los dos lo habría encontrado una revisión de lectura, porque los dos eran **plausibles**. Los encontró una colección de Postman que fue a preguntarle al servicio.